
FACULTY OF ENGINEERING AND TECHNOLOGY

FINAL ASSESSMENT FOR THE

BSC (HONS) INFORMATION TECHNOLOGY; BSC (HONS) COMPUTER SCIENCE; BSC SOFTWARE ENGINEERING (HONS)
YEAR 2

ACADEMIC SESSION 2026; SEMESTER 2

PRG2104: Object-Oriented Programming

Final Project — ScalaFX SDG-1 Application (Tier C — AI-Integrated; individual)

Project deadline: Monday of Week 14, 16:00 (Elearn). Late = 0.

STUDENT DETAILS

STUDENT NAME: _____

STUDENT ID (8 digits): _____

PROGRAMME (IT / CS / SE): _____ **COHORT:** _____

PROJECT TITLE: _____

GitHub Classroom assignment URL (every student accepts here): <https://classroom.github.com/a/6oHuAThS>

Submission folder name (must match exactly): Project_<StudentID>

Optional development tools — GitHub, IntelliJ Local History, OneDrive sync — are NOT marked. Marking is performed entirely against the contents of your submitted ZIP. See the Standard Submission Structure page.

Academic Honesty Acknowledgement

"I declare that this final project contains entirely my own work. I have used AI tools as permitted by the PRG2104 Tier C policy and have logged every significant prompt in Template 2 below. I can explain every line of code I have submitted; if I cannot, I accept that those marks are forfeit. I have cited every external source. I have not collaborated with another student on the code; I have not consulted a senior's previous submission."

Student signature: _____ Date: _____

OVERVIEW

Build a standalone ScalaFX desktop application that addresses a chosen aspect of UN Sustainable Development Goal 1 — “No Poverty.” Demonstrate the advanced OOP and third-party-library skills you have acquired across the semester. Defend your work in a 3-minute rapid viva. <https://classroom.github.com/a/6oHuAThS>

ASSESSING SUBJECT LEARNING OUTCOME

- CLO 4 / SLO: Write computer programs that utilise third-party object-oriented libraries. (C3, PLO 7)

Sub-objectives demonstrated: inheritance, polymorphism (subtype + parametric), abstraction, immutability, exception handling, GUI event-driven programming, and disciplined AI use.

AI USAGE — TIER C (AI-INTEGRATED)**Tier C — AI is expected, logged, declared, and defended in viva**

- AI tools (ChatGPT, Claude, Gemini, GitHub Copilot, Cursor, etc.) are EXPECTED. Use them well.
- Section 4 awards 10 marks for the QUALITY of your AI use — your prompts, your iterative refinement, the way you critiqued and adapted AI output.
- Template 2 (AI Interaction Log) MUST contain at least 10 entries. Each entry: prompt, tool, response summary, what YOU changed, file affected.
- Template 3 (AI Use Declaration) MUST be signed at submission.
- Template 4 (Viva Voce Rubric) is used during the live viva. Viva is MANDATORY — failing to attend awards 0 across the whole project.
- The viva is the anti-AI gate: if you cannot explain a piece of code, those lines do not count toward your mark. Practise walking through your design out loud.

DETAIL OF THE QUESTION — SDG 1 theme

Propose a standalone ScalaFX GUI system that helps achieve SDG 1 (No Poverty) by serving a real-world group experiencing or working against poverty (low-income households, NGOs, community lenders, social workers). Your system MUST:

- Make use of ScalaFX 21. Other GUI libraries are NOT acceptable (automatic zero on UI section).
- Provide AT LEAST FOUR distinct, working features (e.g. add/edit/list/visualise; or login/dashboard/log/report).
- Demonstrate inheritance and polymorphism (subtype + at least one generic class or method).
- Use additional traits, abstract classes, and helper objects to keep code DRY (no repetition).
- Be ORIGINAL — your own design. Do NOT reuse a senior's submission. The marker may compare your code against the previous-year submission library and runs similarity checks on the submitted ZIP.

Example sub-domains (you may pick one, or propose your own with lecturer approval)

Example	What the system does
Microfinance Loan Tracker	Community lenders register borrowers, disburse small loans, schedule repayments, and track default risk with a dashboard view.
Cash-Aid Distribution App	An NGO worker enrolls aid recipients, verifies eligibility, disburses cash transfers, and reports per-village totals.
Subsidy Eligibility Calculator	A household enters income, dependents, and assets; the system determines which government subsidies they qualify for and prints application checklists.
Community Skills Marketplace	People with limited cash income offer skills (tutoring, repairs, childcare) in exchange for credits redeemable for other community services.
Food Pantry Inventory & Demand	A pantry coordinator logs donated items, expiring stock, family requests, and produces a daily distribution plan that minimises waste.
Other (your idea)	Any standalone GUI tool that addresses a specific aspect of SDG 1 (No Poverty) — e.g. emergency-fund savings, scholarship matchmaking, low-income transit-pass tracker. Get lecturer's approval before Week 12 starts.

DEVELOPMENT PLATFORM (required)

Component	Required
Build	sbt 1.9+ (any latest version)
IDE	IntelliJ IDEA recommended; any IDE acceptable
JVM	Java 21
Language	Scala 3.3+
UI Library	ScalaFX 21 (other GUI libraries are NOT acceptable — automatic zero on UI section)
Test (recommended)	ScalaTest 3.2.x or munit 1.x
Persistence (optional)	Plain text/CSV/JSON files; SQLite via JDBC if needed

WHAT YOU HAND IN

ONE ZIP file named Project_<StudentID>.zip containing the standardised submission folder shown on the next page. Every file path inside the folder is fixed — markers and AI-assisted marking tools rely on the exact tree to verify your work efficiently.

- README.md — setup + run instructions; project summary; AI-use summary.
- build.sbt + project/build.properties — sbt build configuration.
- src/main/scala/ — all Scala source code; src/main/resources/ for sample data; src/test/scala/ for any tests.
- docs/ — UML diagram, personal reflection, AI reflection, dev log, citations, demo video.
- ai/ — AI Interaction Log (Template 2) and AI Use Declaration (Template 3).
- submission_manifest.md — filled-in checklist + self-reported metrics; signed by you.

SUBMISSION RULES**Penalties**

- Late submission = 0 marks. No exceptions.
- Originality is checked through docs/dev_log.md (≥ 5 dated entries) and the live viva — you may use any IDE / version-control tool you like, but only your submitted ZIP is marked.
- Cite every external source (YouTube, blog, third-party library) with URL + license. Uncited code = 0 on that part.
- Senior's previous submission is NOT a permitted source.
- Viva is MANDATORY. Missing the viva = 0 across the project.

STANDARD SUBMISSION STRUCTURE (must match exactly)

The marker (or an AI-assisted marking tool) navigates this exact tree to find every artifact. Wrong file names or missing folders forfeit the matching criterion.

```

Project_<StudentID>/
├── README.md                # setup, run, project summary, AI summary
├── build.sbt                # Scala 3.3+ + ScalaFX 21
├── project/
│   └── build.properties     # sbt version
├── src/
│   ├── main/
│   │   ├── scala/          # all production source files
│   │   └── resources/      # sample data, assets, CSV/JSON
│   └── test/
│       └── scala/          # unit tests (optional but recommended)
├── docs/
│   ├── UML.png             # or UML.pdf – class diagram, ≥5 classes
│   ├── reflection.md       # personal reflection (~350-700 words)
│   ├── ai_reflection.md    # AI integration reflection (300-500 words)
│   ├── dev_log.md         # ≥5 dated entries spanning ≥5 calendar dates
│   ├── citations.md        # third-party code/asset citations + licenses
│   └── demo.mp4            # ≤5 min walkthrough of all 4+ features
├── ai/
│   ├── interaction_log.md  # Template 2 – ≥10 entries
│   └── declaration.md      # Template 3 – signed
└── submission_manifest.md  # Checklist + self-reported metrics, signed

```

Why this structure helps marking**What this means for your mark**

- Markers (and AI-assisted marking tools like Cowork) know EXACTLY where to find each artifact — no hunting through a chaotic ZIP.
- Every Section 1–4 criterion in this rubric points at a specific path (e.g. docs/dev_log.md, ai/interaction_log.md, src/main/scala/). The marker can run automated checks (file existence, word count, grep patterns) directly.
- Self-reported metrics in submission_manifest.md let the marker spot-check rather than count from scratch.
- Wrong path = automatic 0 on the criterion that depends on it. Run the file checklist twice before you ZIP.

submission_manifest.md — fill in this template, save it AS **submission_manifest.md**, sign it, drop it at the root of your folder

```
# Submission Manifest – Project_<StudentID>

**Student name:** <full name>
**Student ID:** <8 digits>
**Project title:** <your title>
**Theme:** SDG 1 – No Poverty / sub-domain: <e.g. Microfinance Tracker>
**Date submitted:** <YYYY-MM-DD>

## File checklist (tick every box)

- [ ] README.md                – setup, run, project summary, AI summary
- [ ] build.sbt                 – Scala 3.3+, ScalaFX 21
- [ ] project/build.properties  – sbt version
- [ ] src/main/scala/           – all production source files
- [ ] src/main/resources/       – sample data / assets (if any)
- [ ] src/test/scala/           – unit tests (optional)
- [ ] docs/UML.png (or .pdf)    – class diagram with ≥5 classes
- [ ] docs/reflection.md        – personal reflection (~350–700 words)
- [ ] docs/ai_reflection.md     – AI reflection (300–500 words)
- [ ] docs/dev_log.md          – ≥5 dated entries
- [ ] docs/citations.md        – third-party citations
- [ ] docs/demo.mp4            – ≤5 min walkthrough
- [ ] ai/interaction_log.md     – Template 2 (≥10 entries)
- [ ] ai/declaration.md        – Template 3 (signed)

## Self-reported metrics (will be auto-verified)

| Metric | Required | Your value |
|---|---|---|
| Number of classes / traits in src/main/scala | ≥ 5 | ____ |
| Number of `// ai-assisted: #N` tags in src/ | ≥ 3 | ____ |
| Number of entries in ai/interaction_log.md | ≥ 10 | ____ |
| Distinct dates in docs/dev_log.md | ≥ 5 | ____ |
| Word count of docs/reflection.md | 350–700 | ____ |
| Word count of docs/ai_reflection.md | 300–500 | ____ |
| sbt -Wunused clean compile passes? | Yes | ____ |
| All four required features work end-to-end? | Yes | ____ |

## Signature

I confirm the above is accurate and that the contents of this folder
are entirely my own work, prepared under the Tier C AI-Integrated
policy.

Signed: _____ Date: _____
```

PROJECT TIMELINE

Week	Phase	Expected work
11	Brief & plan	Read brief; pick SDG-1 sub-domain; sketch UML on paper; create your local Project_<StudentID>/ folder following the Standard Submission Structure; write a one-page README naming theme + at least 4 features; create dev_log.md and add the first dated entry. Begin AI Interaction Log on Day 1.
12 (early)	Build core	Implement domain model (case classes, sealed traits, generics). Wire ScalaFX skeleton (JFXApp3, ≥4 screens, navigation). Open feature branches; tag commits with log-entry numbers when AI assisted.
12 (mid)	Polish + test	Add input validation, exception handling, persistence. Take demo video. Compile with -Wunused; resolve all warnings.
12 (late)	Document + submit	UML diagram final; personal reflection; AI Interaction Log polished (≥10 entries); Use Declaration signed; ZIP and submit on Elearn. Sign up for viva slot.
13 - 14 (viva day)	Viva Voce	3-minute rapid viva: marker picks one class + one // ai-assisted block at random; you answer three 60-second prompts (purpose & design / line-by-line walk-through / weakness & fix). Template 4 scored on the spot.

Section 1 — Application & OOP Design (20 marks)

Each row is one mark. Award 1 if the criterion is true, 0 otherwise. Sum and read the band off the table at the end of this section.

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
Submission structure, build & originality (5 marks)			
S1-1	Submission folder follows the standard structure: README.md, build.sbt, project/, src/, docs/, ai/, submission_manifest.md ALL present at the expected paths.	ls Project_<StudentID>/ → 6 expected entries appear. See “Standard Submission Structure” page for full tree.	1
S1-2	Project compiles cleanly: sbt clean compile under Scala 3.3+ with -Wunused, zero warnings.	cd Project_<StudentID> && sbt -Wunused clean compile → exit code 0, no [warn] lines.	1
S1-3	Originality evidence: docs/dev_log.md contains ≥ 5 dated entries spanning ≥ 5 distinct calendar dates.	Open docs/dev_log.md; count lines starting with a date heading (e.g. ## 2026-04-22 or -[2026-04-22]); distinct dates ≥ 5.	1
S1-4	All third-party / external code or assets cited (URL + license) in docs/citations.md AND tally matches resources used.	grep -E '(http License MIT Apache GPL BSD)' docs/citations.md → count ≥ 1 per cited resource; non-trivial external files have a header comment pointing to citations.md.	1
S1-5	Naming: PascalCase types, camelCase methods, no single-letter identifiers in production code.	grep -E '(class trait object) [a-z]' → zero hits. grep -E 'val [a-z] var [a-z]' → zero hits in src/main/scala.	1
OOP design (8 marks)			
S1-6	At least 5 distinct classes / traits in src/main/scala defined by the student.	grep -E '^(class trait abstract class sealed trait case class enum) ' → count ≥ 5.	1
S1-7	Inheritance: ≥ 1 abstract class OR sealed trait with ≥ 2 concrete subclasses extending it.	grep -E '(abstract class sealed trait) (\w+)' AND grep -E 'extends (\w+)' for the same name ≥ 2.	1
S1-8	Subtype polymorphism: ≥ 1 explicit override of an inherited method.	grep 'override ' → count ≥ 1 in src/.	1
S1-9	Parametric polymorphism: ≥ 1 generic class or generic method [T] (or [A], [B] ...).	grep -E '(class trait def) \w+\[[A-Z]' → count ≥ 1.	1
S1-10	Encapsulation: ≥ 1 private OR protected field, with rationale comment.	grep -E '(private protected) (val var)' → count ≥ 1, AND adjacent // comment.	1
S1-11	Immutability: zero var, zero mutable.* collections in domain code.	grep -wE 'var ArrayBuffer ListBuffer mutable\. Array\[' → zero hits in src/main/scala (excluding ScalaFX bindings).	1
S1-12	Exception handling: every IO or risky op wrapped in scala.util.Try OR Either OR Option.	Every Source.fromFile / parse / toInt etc. is wrapped; no naked 'throw new' or unchecked .get on Option.	1
S1-13	DRY: a Common base trait, abstract class, or helper object extracts shared logic (no copy-paste).	Marker reads the source for evident shared base.	1
Functional features (4 marks — 1 per feature, max 4)			
S1-14	Feature 1 works end-to-end (input → process → output / persistence).	Demo video shows F1; data persists if applicable.	1
S1-15	Feature 2 works end-to-end and is meaningfully different from F1.	Demo video shows F2 distinct from F1.	1
S1-16	Feature 3 works end-to-end.	Demo video shows F3.	1

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
S1-17	Feature 4 works end-to-end.	Demo video shows F4.	1
Robustness (3 marks)			
S1-18	At least 3 invalid-input cases handled gracefully (no stack traces shown to user).	Marker tries: empty input, wrong type, out-of-range. Each shows a friendly error message in the UI.	1
S1-19	Application does not crash during the marker's 5-minute exploration.	No uncaught exception during demo.	1
S1-20	Persistence (file or DB) survives application restart for at least one feature.	Marker adds an item, restarts the app, item is still there.	1
TOTAL (sum of awarded marks)			/ 20

Section 1 — total-to-band mapping

MARGINAL (D)	ADEQUATE (C)	GOOD (B)	EXCELLENT (A)
0 – 7 marks	8 – 11 marks	12 – 15 marks	16 – 20 marks
Marginal	Adequate	Good	Excellent

Section 2 — ScalaFX UI (10 marks)

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
S2-1	Application launches into a ScalaFX JFXApp3 main window without crash.	Run <code>sbt run</code> ; window appears within 10 seconds; no unhandled exception in <code>stderr</code> .	1
S2-2	At least 4 distinct UI screens / panes accessible from the main window.	Demo video walks through ≥ 4 named screens.	1
S2-3	Top-bar menu OR side-bar navigation works between all screens.	Marker clicks each menu item; expected screen appears.	1
S2-4	At least 4 distinct ScalaFX controls used (TextField, Button, ListView, ComboBox, TableView, ...).	<code>grep scalafx.scene.control.* imports OR usages</code> → <code>count</code> ≥ 4 .	1
S2-5	Layout uses ScalaFX layout containers (BorderPane / VBox / HBox / GridPane), no raw absolute pixel positioning.	<code>grep scalafx.scene.layout.*</code> → <code>count</code> ≥ 1 ; no <code>setLayoutX/setLayoutY</code> for primary layout.	1
S2-6	Event handlers use ScalaFX idiom: <code>onAction = handle { ... }</code> (NOT Java's <code>setOnAction(EventHandler{...})</code>).	<code>grep 'onAction = handle'</code> <code>count</code> ≥ 1 ; <code>grep 'setOnAction('</code> should be zero in student code.	1
S2-7	Reactive properties demonstrated (ObjectProperty, StringProperty, ObservableBuffer / ObservableList).	<code>grep -E 'ObjectProperty StringProperty ObservableBuffer ObservableList'</code> → <code>count</code> ≥ 1 .	1
S2-8	Imports use <code>scalafx.*</code> (NOT <code>javafx.*</code> directly for controls / layout).	<code>grep 'import javafx'</code> should not appear for controls/layout (only <code>javafx.event</code> types are tolerated).	1
S2-9	Visually polished: consistent colours, padding ≥ 8 px between sections, no overlapping widgets.	Marker eyeballs the running app; subjective but uniform pad/colour visible.	1
S2-10	Application is usable from keyboard: tab order works, Enter submits the focused form.	Marker tries <code>tab + enter</code> only; form submits.	1
TOTAL (sum of awarded marks)			/ 10

Section 2 — total-to-band mapping

MARGINAL (D)	ADEQUATE (C)	GOOD (B)	EXCELLENT (A)
0 – 3 marks	4 – 5 marks	6 – 7 marks	8 – 10 marks
Marginal	Adequate	Good	Excellent

Section 3 — Documentation (5 marks)

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
S3-1	UML class diagram present, legible, shows ≥ 5 classes WITH relationships (inheritance / association arrows).	Open UML.png/UML.pdf in /docs; count distinct classes ≥ 5 ; arrows show is-a / has-a.	1
S3-2	README has setup + run instructions a stranger can follow (sbt commands, dependencies, sample run).	Marker follows README on a clean machine; sbt run succeeds.	1
S3-3	Personal reflection (~1 A4 page) covers OOP applied + problems + strengths/weaknesses (3 sub-points required).	wc -w of reflection $\in [350, 700]$; 3 sub-points are explicitly labelled.	1
S3-4	Demo video (≤ 5 min) walks through ALL 4+ features end-to-end.	Open the .mp4; verify all features named in README appear in the video.	1
S3-5	All third-party code / assets cited with URL + license in README (or code comment) AND tally matches assets in repo.	grep -E 'License MIT Apache GPL BSD' README.md; non-trivial external files have a citation header.	1
TOTAL (sum of awarded marks)			/ 5

Section 3 — total-to-band mapping

MARGINAL (D)	ADEQUATE (C)	GOOD (B)	EXCELLENT (A)
0 – 1 mark	2 marks	3 marks	4 – 5 marks
Marginal	Adequate	Good	Excellent

Section 4 — AI Integration (10 marks)

This is where Tier C earns its weight. We assess HOW WELL you used AI, not whether you used it. Quality, not volume.

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
S4-1	Template 2 has at least 10 fully-completed entries.	Count rows in Template 2 with all 6 columns non-empty.	1
S4-2	At least 3 entries demonstrate ITERATIVE prompting (multi-turn refinement, not one-shot).	≥ 3 rows reference an earlier entry by # OR include phrases like 're-prompted', 'refined', 'asked again'.	1
S4-3	At least 5 entries explicitly state what YOU CHANGED in the AI output (not 'accepted verbatim').	'What YOU changed' column for ≥ 5 rows contains specific edits ('renamed X', 'replaced match with sealed trait', 'rewrote helper').	1
S4-4	Prompt quality: at least 5 prompts include domain context (Scala 3 / ScalaFX / SDG-1 specifics / your data model).	Read the Prompt column for 5 rows; each names a specific class/feature, the language version, or the constraint.	1
S4-5	AI-assisted code is tagged in source: <code>// ai-assisted: log-#N</code> for every block written with AI help.	<code>grep -E '// *ai-assisted: *#\d+' in src/</code> → count ≥ 3 distinct entry numbers referenced.	1
S4-6	Each <code>// ai-assisted: #N</code> tag in source has a sibling <code>// why: <reason></code> comment within 2 lines explaining why AI was used for that block.	For every <code>// ai-assisted: #N</code> hit, look at the next 2 lines for <code>// why;</code> ; ratio of tagged blocks WITH a why-comment ÷ total tagged blocks ≥ 0.80.	1
S4-7	Template 3 (AI Use Declaration) is signed AND lists every distinct AI tool that appears in Template 2.	Signature filled; tool list = set(distinct tools in Template 2).	1
S4-8	AI Reflection (300–500 words) covers ALL three required questions (most useful / misled / what AI couldn't do).	Reflection has 3 named paragraphs OR 3 sub-headings; word count ∈ [300, 500].	1
S4-9	AI Reflection cites at least 3 specific Template-2 entry numbers.	<code>grep -Eo '#?\b\d+\b'</code> across reflection → ≥ 3 numbers in the range 1..(entries).	1
S4-10	Critical evaluation: at least 1 entry honestly admits an AI hallucination or wrong suggestion AND how the student spotted it.	≥ 1 row in 'AI response summary' or 'What you changed' uses words like 'wrong', 'hallucinated', 'didn't compile', 'reverted'.	1
TOTAL (sum of awarded marks)			/ 10

Section 4 — total-to-band mapping

MARGINAL (D)	ADEQUATE (C)	GOOD (B)	EXCELLENT (A)
0 – 3 marks	4 – 5 marks	6 – 7 marks	8 – 10 marks
Marginal	Adequate	Good	Excellent

Section 5 — Viva Voce (Template 4) (5 marks — 3-minute rapid format)

Three minutes. Three rapid-fire questions. Five binary checks. The viva is the anti-AI gate: if a student cannot explain a piece of code in their submission, the marker may withhold the corresponding marks from Sections 1, 2 and 4 in addition to awarding 0 here.

Procedure (marker reads this once, follows it for every student)

How to run a 3-minute viva

- BEFORE the viva (≈30 seconds): open the student's repo on a second monitor. Pick ONE class from src/ at random — write its name on the marking sheet. Pick ONE block tagged // ai-assisted: #N at random — note the file, line range, and matching Template-2 entry number.
- PHASE 1 — 60 seconds. Show the student the class name. Ask: "What does this class do, and what is one design decision you made about it that you might have made differently?"
- PHASE 2 — 60 seconds. Show the student the // ai-assisted block. Ask: "Walk me through this block, line by line." Do NOT prompt — let silence be silence.
- PHASE 3 — 60 seconds. Ask: "What is the single weakest part of your design, and how would you fix it given another week?"
- AFTER the viva (≈30 seconds): tick the five S5 criteria below; record total in Final Mark Summary. If a critical answer is missing, briefly note which Sections 1/2/4 marks are now in doubt.
- Borderline cases only: if the marker is genuinely unsure after 3 minutes, ONE follow-up question is permitted (extending the slot to ~5 minutes). Use this sparingly — fewer than 1 in 10 students.

Marker score sheet (tick during and immediately after the 3 minutes)

#	Criterion (must be true to award the mark)	Evidence — what the marker checks	Marks
S5-1	Phase 1 — student articulates the picked class's purpose in ONE clear sentence within 30 seconds.	Listen for a single noun-phrase + verb sentence. "It's the class that handles X for Y." If the student rambles for >30s without naming a purpose, mark 0.	1
S5-2	Phase 1 — student names ONE specific design decision in that class AND ONE concrete alternative they considered.	Decision must be specific (e.g. "made it a sealed trait", "used Option not null"). Alternative must be concrete (e.g. "could have used inheritance"). Both required.	1
S5-3	Phase 2 — student walks through the // ai-assisted block fluently: ≤ 5 seconds of silence per line, no Googling, no re-reading file from scratch.	If the student pauses > 5 seconds without speaking, or visibly searches the web / asks the AI, mark 0. Reading the line aloud and explaining its role counts as fluent.	1
S5-4	Phase 3 — student names a REAL, specific weakness in their design (not a generic platitude).	Specific: "My X class is doing too much; the Y feature should be its own class." Generic / unacceptable: "More tests would be nice.", "More features.", "Better UI.".	1
S5-5	Phase 3 — student offers a CONCRETE, technically valid fix for that weakness.	Fix names a Scala / OOP construct (extract trait, push behaviour to subclass, replace var with State + transition, introduce a generic). Vague fixes ("refactor", "clean up") do not count.	1
TOTAL (sum of awarded marks)			/ 5

Section 5 — total-to-band mapping

MARGINAL (D)	ADEQUATE (C)	GOOD (B)	EXCELLENT (A)
0 – 1 mark	2 marks	3 marks	4 – 5 marks
Marginal	Adequate	Good	Excellent

Why 3 minutes works**Rationale (lecturer reference, not for student)**

- 120 students $\times$ 3 min $\approx$ 6 hours of viva time, distributable across two sittings or two markers.
- Random selection of class + block means students cannot rehearse a single answer; they must understand the WHOLE project.
- Three focused 60-second prompts catch the same signal as a 10-minute walk-through: can the student think on their feet about THEIR OWN code?
- Live modification has been removed — it added time but rarely changed the score in a 5-band rubric.
- Borderline-case follow-up question keeps marker time bounded at $\sim$ 3 min for $\sim$ 90% of students; $\sim$ 5 min for the unclear $\sim$ 10%.

Template 2 — AI Interaction Log (minimum 10 entries)

Log every significant AI prompt that influenced your code or your write-up. Each row is one prompt. If you ran the same prompt against multiple tools, log each as a separate row. Add rows below the table if you exceed 10.

#	Prompt you sent	AI tool	AI response — short summary	What YOU changed (paraphrase / fix / accept verbatim)	Module / file
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					

Template 3 — AI Use Declaration

I, the student named on the cover page, declare that:

1. I used AI tools as permitted by the PRG2104 Tier C policy. Every significant prompt is logged in Template 2 above.
2. I can EXPLAIN every line of code I have submitted. If I cannot, I accept that those marks are forfeit during the viva.
3. AI-generated code blocks are tagged in source with `// ai-assisted: #N` referencing the matching Template 2 entry.
4. I did NOT consult a senior's previous submission for this project, nor did I share my submission with another student.
5. I understand that fabricated log entries, undeclared AI use, or inability to defend submitted code in the viva is academic misconduct under the Sunway University Academic Integrity Policy.

AI tools used (list every distinct tool from Template 2):

Student signature: _____ Date: _____

Submitting without a signed declaration voids the AI-integration component AND blocks the viva.

AI INTEGRATION REFLECTION (300–500 words; assessed in S4-8 and S4-9)

Answer all three of the following in your reflection. Cite specific Template-2 entry numbers (e.g. “entry #4 was where I first noticed the hallucination”).

- What was AI most useful for? Cite ONE specific Template-2 entry by number.
- Where did AI mislead you, hallucinate, or produce wrong Scala / ScalaFX? Cite ONE specific entry.
- What did YOU do that AI could NOT have done for you? (Reasoning about the data, choosing the design, defending it in viva.)

Write your reflection on a separate A4 page and attach it to your submission.

FINAL MARK SUMMARY (for marker)

Section	Awarded	Out of	Band
Section 1 — Application & OOP Design	_____	20	_____
Section 2 — ScalaFX UI	_____	10	_____
Section 3 — Documentation	_____	5	_____
Section 4 — AI Integration	_____	10	_____
Section 5 — Viva Voce (Template 4)	_____	5	_____
TOTAL (Final Project = 50% of final grade)	_____	50	_____

Final letter grade follows the standard Sunway University A / B / C / D bands applied to the percentage.

MARKER TIME-BUDGET (per student, target ~19 minutes)

- 5 min — open repo, run sbt clean compile, run app, watch demo video while it builds (offline check).
- 8 min — tick Sections 1–3 criteria using grep + visual checks against the criteria tables (offline check).
- 3 min — tick Section 4 (AI integration) using Template 2 + Template 3 + reflection (offline check).
- 3 min — viva: 30 s prep + 3 × 60 s phases + 30 s scoring (live with student, see Section 5).

Total per student: ~19 minutes (16 min offline + 3 min viva). 120 students × 19 min ≈ 38 hours total marking — comfortably distributable across two markers or two weeks.

SUBMISSION CHECKLIST (student fills before turn-in)

- Submission folder named Project_<StudentID>/ matches the Standard Submission Structure exactly (see folder-tree page).
- submission_manifest.md filled in and signed; every checkbox ticked; self-reported metrics filled.
- README.md has setup + run instructions a stranger can follow on a clean machine.
- docs/ contains UML.{png,pdf}, reflection.md, ai_reflection.md, dev_log.md (≥5 dated entries), citations.md, demo.mp4.
- ai/ contains interaction_log.md (≥10 entries) and declaration.md (signed).
- Project compiles with sbt -Wunused clean compile, zero warnings.
- ZIP the Project_<StudentID> folder and TURN IN on Elearn BEFORE Monday Week 14, 16:00.
- Viva slot booked through the lecturer's Calendly.